Analyzing Huffman Coding for Lossless Text Compression

Project Report

Submitted by:

Manish Sinha (22ECE1018)

Ajay Yadav (22ECE1041)

Under the Guidance of:

Dr. Trilochan Panigrahi

Associate Professor

Department of Electronic & Communication Engineering

National Institute of Technology Goa

Cuncolim, South Goa, Goa 403703

ACKNOWLEDGEMENT

It is with great respect and sincere appreciation that I acknowledge the Department of Electronics and Communication Engineering, National Institute of Technology Goa, for providing me the opportunity to undertake this minor project under the *Digital Communication* course.
I am deeply grateful to **Dr. Trilochan Panigrahi** for his invaluable guidance, constructive feedback, and steadfast support throughout the course of this project. His insightful suggestions and encouragement played a pivotal role in shaping the direction and successful completion of this academic endeavor.

I extend my sincere thanks to the faculty members and technical staff of the department for their assistance and for fostering an environment of academic collaboration and continuous learning.

Finally, I wish to express my heartfelt gratitude to my family and well-wishers, whose unwavering support and motivation have been a constant

Contents

ABSTRACT……4

1. INTRODUCTION……5
   - 1.1 Background……5
   - 1.2 Problem Statement……5
   - 1.3 Objectives……5
2. METHODOLOGY……6
   - 2.1 Approach……6
   - 2.2 Implementation……6
     - 2.2.1 Frequency Analysis……6
     - 2.2.2 Constructing the Huffman tree……7
     - 2.2.3 Assigning Binary Codes to Symbols……7
     - 2.2.4 Encoding the Data……8
     - 2.2.5 Decoding the Encoded Data……8
     - 2.2.6 Analysing the Compression……9
     - 2.2.7 Output and Results……9
   - 2.3 Code: Python Implementation……10
     - 2.3.1 Python Code & Explanation……10
3. RESULT AND ANALYSIS……16
   - 3.1 Sample Text……16
   - 3.2 Results……16
   - 3.3 Example Output……16
4. CONCLUSION……20
5. FUTURE SCOPE……21
6. BIBLIOGRAPHY……23

ABSTRACT

Huffman coding is a lossless data compression algorithm that assigns variable-length binary codes to characters based on their frequencies in the input text. Characters occurring more frequently are assigned shorter codes, which leads to a reduction in the overall size of the data. This project implements Huffman coding in Python, demonstrating its application on a sample input text and evaluating the resulting compression.

The process begins with calculating character frequencies, which are then used to build a binary Huffman tree via a priority queue (min-heap). The tree is traversed to generate unique, prefix-free binary codes for each character. The original message is encoded using these codes, and the binary string is then decoded using the tree to verify accuracy.

The project evaluates the compression efficiency by comparing the original and compressed bit sizes and analysing metrics like entropy and average code length. The results show that Huffman coding achieves good compression performance, especially on texts with non-uniform character distributions. This highlights its effectiveness for storage and transmission in digital systems.

INTRODUCTION

1.1 Background

With the exponential growth of digital data in recent years, efficient data compression has become essential in computing and communications. Lossless compression techniques, which preserve the original data exactly after decompression, are crucial in areas like text file storage, image processing, and data transmission. Among the most prominent algorithms for lossless compression is **Huffman coding**, proposed by David A. Huffman in 1952. This method is optimal for symbol-based compression where frequencies are known, making it foundational in both academic study and practical applications such as ZIP compression and multimedia codecs like JPEG and MP3.

1.2 Problem Statement

Conventional encoding schemes like fixed-length ASCII represent each character using the same number of bits, regardless of frequency of occurrence. This leads to inefficiencies, especially when some characters appear far more frequently than others. The challenge is to find an encoding that uses fewer bits for common characters and more bits for rarer ones—without losing any information. Huffman coding addresses this issue by creating a variable-length prefix code that reflects the probability distribution of characters.

1.3 Objectives

- To understand and implement the Huffman coding algorithm using Python.
- To build a Huffman tree based on character frequencies in a given text.
- To generate efficient binary codes and compare the compressed size with the original size.
- To evaluate performance using entropy, average code length, and compression ratio.
- To visualize the results and verify lossless decoding.

METHODOLOGY

2.1 Approach

The methodology consists of the following key steps:

1. **Frequency Analysis**: Count the number of occurrences of each character.
2. **Tree Construction**: Use a min-heap to iteratively build the Huffman tree by merging the lowest-frequency nodes.
3. **Code Generation**: Traverse the tree to assign binary codes to characters.
4. **Encoding**: Replace each character in the original text with its binary Huffman code.
5. **Decoding**: Reconstruct the original text from the encoded string using the tree.
6. **Analysis**: Compute entropy, average code length, and compression ratio for evaluation.

2.2 Implementation

Huffman coding is a **lossless data compression algorithm** that assigns variable-length codes to input characters based on their frequencies. The goal is to represent frequent characters with shorter codes and rare characters with longer codes, minimizing the total number of bits required for encoding.

2.2.1: Frequency Analysis

Before encoding, it's essential to analyze the frequency of each symbol in the dataset. The frequency of each symbol indicates how often it appears in the dataset. This frequency is used to determine the length of the binary code each symbol will be assigned.

Process:

- **Character Frequencies**: For each symbol (character) in the dataset, the algorithm counts how often it appears.
- **Example of Frequency Count**: In a dataset with 10 symbols, if one symbol appears 4 times and another appears 6 times, the algorithm knows that the more frequent symbol should get a shorter binary code.

2.2.2: Constructing the Huffman Tree

The next step is to build the **Huffman Tree** using the frequencies of the symbols. The Huffman tree is a binary tree where:

- Each leaf node represents a character and its frequency.
- Internal nodes represent the sum of the frequencies of their child nodes.

Process of Tree Construction:

1. **Create Leaf Nodes**: Start by creating a node for each symbol, where each node contains the character and its frequency.
2. **Priority Queue**: All the leaf nodes are placed into a **min-heap** (priority queue) so that nodes with smaller frequencies are given higher priority.
3. **Merging Nodes**: Repeatedly extract the two nodes with the smallest frequencies, create a new internal node with a frequency equal to the sum of the two nodes, and insert this new node back into the heap.
4. **Repeat**: This process continues until only one node remains, which will be the root of the Huffman tree. This final node represents the entire dataset, and its left and right children represent the smaller merged frequencies.

At the end of this process, the Huffman tree is built with the root node representing the entire set of symbols, and the leaf nodes representing the individual characters.

2.2.3: Assigning Binary Codes to Symbols

Once the tree is built, the next step is to assign **binary codes** to the characters based on their position in the tree.

Process of Code Assignment:

1. **Depth-First Traversal**: Starting from the root of the Huffman tree, traverse the tree depth-first.
2. **Left and Right Movement**: Move left in the tree and append a '0' to the current code. Move right and append a '1'.
3. **Leaf Node Encoding**: When a leaf node is reached (which contains a character), the accumulated code along the path from the root to the leaf becomes the **Huffman code** for that character.

4. **Prefix Property**: Since the algorithm ensures that no code is a prefix of another, we guarantee that the binary codes are uniquely decodable. This property allows for efficient compression and reliable decoding.

2.2.4: Encoding the Data

With the Huffman codes for each character determined, the next step is to **encode** the original dataset. The encoding process replaces each character in the dataset with its corresponding Huffman code.

Process of Encoding:

1. **Substitute Characters**: Each character in the input dataset is replaced by its binary Huffman code.
2. **Concatenate the Codes**: The individual Huffman codes are concatenated to form the final encoded binary string.

The resulting binary string is the **compressed** representation of the original dataset, which will be much shorter if the dataset has many frequently occurring characters.

2.2.5: Decoding the Encoded Data

The encoded data can be decoded back to its original form using the same Huffman tree.

Process of Decoding:

1. **Start at the Root**: Begin decoding from the root of the Huffman tree.
2. **Traverse the Tree**: For each bit in the encoded binary string:
   - Move left in the tree for a '0'.
   - Move right for a '1'.
3. **Leaf Node Reached**: When a leaf node (character) is reached, append the character to the decoded string and reset to the root of the tree to begin decoding the next symbol.
4. **Complete Decoding**: This process continues until the entire encoded binary string is decoded back to the original sequence of characters.

2.2.6: Analyzing the Compression

After encoding and decoding, we need to assess the efficiency of the compression:

1. **Compression Ratio**: This ratio compares the size of the original dataset (in bits) to the size of the encoded dataset (in bits). A high compression ratio indicates effective compression.

$$\text{Compression Ratio} = \frac{\text{Original Size (bits)}}{\text{Compressed Size (bits)}}$$

2. **Entropy:** The entropy of the dataset measures the minimum number of bits required to represent each symbol, assuming an optimal encoding scheme. The closer the average length of the Huffman codes is to the entropy, the better the compression.

$$H(X) = -\sum_{i=1}^{n} p_i \log_2(p_i)$$

Where p_i is the probability (frequency) of each symbol.

3. **Average Code Length**: This is the average number of bits used to represent each symbol, weighted by the frequency of the symbols. It provides an estimate of how efficiently the data has been compressed.

$$\text{Average Code Length} = \sum_{i=1}^{n} p_i \times \text{length of code for symbol } i$$

2.2.7: Output and Results

Finally, after the encoding and decoding processes are completed, the results include:

1. The **original data** (which was encoded).
2. The **frequencies** of the symbols in the dataset.

3. The **Huffman codes** assigned to each symbol.
4. The **encoded binary string**.
5. The **decoded string**, which should match the original data.
6. **Compression statistics** such as:
    - **Compression ratio**: Indicates how much the data has been compressed.
    - **Entropy**: The theoretical lower bound for the compression.
    - **Average code length**: How efficiently the symbols are encoded.

2.3 Code: Python Implementation

The Python implementation of Huffman Coding compresses data by assigning shorter binary codes to frequently occurring characters and longer codes to less frequent ones. It starts by calculating character frequencies from the input text. These frequencies are used to build a min-heap, where each character is wrapped in a node. The two nodes with the lowest frequencies are repeatedly merged to build a binary Huffman Tree.

After the tree is built, a recursive traversal assigns binary codes to each character. Encoding replaces each character with its binary code, while decoding uses the tree to translate the binary string back into the original text. The implementation also calculates entropy, average code length, and compression ratio, providing insight into the efficiency of the compression.

2.3.1 PYTHON CODE

1. Importing Required Modules

```
import heapq
from collections import defaultdict, Counter
import math
```

Explanation:

- heapq: Provides a priority queue (min-heap) implementation to efficiently build the Huffman Tree.
- Counter: Helps in counting character frequencies in the input text.
- math: Used for entropy calculation using logarithmic functions.

2. Node Class for Huffman Tree

```
# Node class for Huffman tree
class Node:
    def _init_(self, char=None, freq=0):
        self.char = char
        self.freq = freq
        self.left = None
        self.right = None

    # For priority queue (min-heap)
    def _lt_(self, other):
        return self.freq < other.freq
```

Explanation:

- Node: Represents a node in the Huffman Tree.
- char: Stores the character (only in leaf nodes).
- freq: Frequency of the character or combined frequency for internal nodes.
- left, right: Children of the node in the tree.
- __lt__: Tells Python how to compare nodes in the priority queue (based on frequency).

3. Build the Huffman Tree

```
# Build Huffman Tree
def build_huffman_tree(frequency):
    heap = [Node(char, freq) for char, freq in frequency.items()]
    heapq.heapify(heap)

    while len(heap) > 1:
        node1 = heapq.heappop(heap)
        node2 = heapq.heappop(heap)
        merged = Node(None, node1.freq + node2.freq)
        merged.left = node1
        merged.right = node2

        heapq.heappush(heap, merged)

    return heap[0]
```

Explanation:

- Creates nodes from character frequencies.
- Uses a **min-heap** to repeatedly combine the two nodes with the lowest frequencies into a new internal node.
- This process continues until only one node (the root) is left—representing the full Huffman tree.

4. Generate Huffman Codes

```
# Generate Huffman Codes
def generate_codes(root, code="", codebook={}):
    if root is None:
        return

    if root.char is not None:
        codebook[root.char] = code
        return

    generate_codes(root.left, code + "0", codebook)
    generate_codes(root.right, code + "1", codebook)

    return codebook
```

Explanation:

- Recursively traverses the tree:
  - Left adds a "0" to the code.
  - Right adds a "1".
- Leaf nodes get their code added to the codebook.
- This ensures **prefix-free binary codes**.

5. Encode the Text

```
# Encode the text using Huffman codes
def huffman_encode(text, codebook):
    return ''.join(codebook[char] for char in text)
```

Explanation:

- Replaces each character in the text with its Huffman code from the codebook.

6. Decode the Encoded Text

```
# Decode the binary string using Huffman tree
def huffman_decode(encoded_text, root):
    decoded = ""
    node = root
    for bit in encoded_text:
        node = node.left if bit == "0" else node.right
        if node.char is not None:
            decoded += node.char
            node = root
    return decoded
```

Explanation:

- Follows the bits (0 or 1) to navigate the tree.
- When a leaf node is reached (contains a character), it's added to the output string.
- Then it restarts from the root to decode the next symbol.

7. Calculate Entropy

```
# Calculate entropy
def calculate_entropy(frequency, total):
    return sum((freq / total) * math.log2(total / freq) for freq in
frequency.values())
```

Explanation:

- Entropy measures the minimum number of bits per symbol needed to encode the text.

8.Main Function

```
# Main function
def huffman_coding(text):
    print(f"Original Text:\n{text}\n")
    frequency = Counter(text)
    total_chars = len(text)

    print("Character Frequencies:")
    for char, freq in frequency.items():
        print(f"'{char}': {freq}")

    root = build_huffman_tree(frequency)
    codebook = generate_codes(root)

    print("\nHuffman Codes:")
    for char, code in codebook.items():
```

```
        print(f"'{char}': {code}")

    encoded = huffman_encode(text, codebook)
    decoded = huffman_decode(encoded, root)

    original_bits = total_chars * 8  # assuming 8-bit ASCII
    compressed_bits = len(encoded)

    print(f"\nEncoded Binary:\n{encoded}")
    print(f"\nDecoded Text:\n{decoded}")
    print("\n--- Compression Analysis ---")
    print(f"Original Size: {original_bits} bits")
    print(f"Compressed Size: {compressed_bits} bits")
    print(f"Compression Ratio: {original_bits / compressed_bits:.2f}")
    print(f"Entropy: {calculate_entropy(frequency, total_chars):.4f}
bits/symbol")

    avg_code_length = sum(len(codebook[c]) * (f / total_chars) for c, f in
frequency.items())
    print(f"Average Code Length: {avg_code_length:.4f} bits/symbol")
```

Explanation:

- **Original Text Display**: Prints the original input text.
- **Character Frequency Calculation**: Uses Counter to calculate the frequency of each character in the text.
- **Character Frequency Display**: Prints each character and its frequency.
- **Huffman Tree Construction**: Builds the Huffman tree using the character frequencies.
- **Huffman Code Generation**: Generates a codebook (mapping characters to binary Huffman codes) from the Huffman tree.
- **Huffman Codes Display**: Prints the Huffman codes for each character.
- **Text Encoding**: Encodes the original text using the Huffman codes, resulting in a compressed binary string.
- **Text Decoding**: Decodes the compressed binary string back to the original text to verify correctness.
- **Compression Analysis**:
- Calculates the original size (in bits) and the compressed size.
- Computes the compression ratio (original size / compressed size).
    - **Entropy Calculation**: Calculates the entropy, representing the theoretical minimum number of bits required for the text.
    - **Average Code Length Calculation**: Computes the average length of the Huffman codes based on character frequencies.

9.Example usage

```
# Example usage
if _name_ == "_main_":
    sample_text = "This is Digital Communication Project on Source Coding using Huffman Coding done by Ajay Yadav and Manish Sinha."
    huffman_coding(sample_text)
```

Explanation:

- **Check for Script Execution**:
- if __name__ == "__main__": ensures the code only runs when the script is executed directly.
- **Sample Input Text**:
- Defines a sample_text string about a digital communication project.
- **Call to huffman_coding**:
- huffman_coding(sample_text) calls the function, which:
    - Calculates character frequencies.
    - Builds the Huffman tree.
    - Generates Huffman codes.
    - Encodes and decodes the text.
    - Displays compression analysis, entropy, and average code length.

RESULT AND ANALYSIS

When running the Huffman coding algorithm on the sample input text:

3.1 Sample Text:
"This is Digital Communication Project on Source Coding using Huffman Coding done by Ajay Yadav and Manish Sinha."

3.2 Results:

1. **Original Text:**
    - The original input text is printed to show what is being processed.
2. **Character Frequencies:**
    - The frequency of each character in the input text is displayed. This helps to understand the distribution of characters in the text and is used for constructing the Huffman tree.

3.3 Example output:

Original Text:
This is Digital Communication Project on Source Coding using Huffman Coding done by Ajay Yadav and Manish Sinha.

Character Frequencies:

'T': 1

'h': 3

'i': 11

's': 4

' ': 15

'D': 1

'g': 4

't': 3

'a': 9

'l': 1

'C': 3

'o': 8

'm': 3

'u': 4

'n': 11

'c': 3

'P': 1

'r': 2

'j': 2

'e': 3

'S': 2

'd': 5

'H': 1

'f': 2

'b': 1

'y': 2

'A': 1

'Y': 1

'v': 1

' ': 2

'M': 1

'.': 1

Huffman Codes:

'n': 000

'i': 001

'e': 01000

't': 01001

'.': 010100

' ': 010101

'C': 01011

'm': 01100

'h': 01101

'c': 01110

's': 01111

' ': 100

'S': 101000

'j': 101001

'r': 101010

'b': 1010110

'H': 1010111

'M': 1011000

'v': 1011001

'A': 1011010

'D': 1011011

'l': 1011100

'P': 1011101

'y': 101111

'g': 11000

'u': 11001

'o': 1101

'a': 1110

'Y': 1111000

'T': 1111001

'f': 111101

'd': 11111

Encoded Binary:

1111001011010010111110000101111100101101100111000001010011110101110010001011110101100011001100100000101110111001001001110100010010111011010101101101010010100001110010011001101000100101000110111001101010011100100010001011111011111001000110001001100101111100100011000100101011111001111101111101011001110000100010111101111110010001100010011111110100001000100101011011010111100101101010100111101011111001111000111011111111010110011001110000111010101101100011100000010111101101010101101000001000011011110010100

Decoded Text:

This is Digital Communication Project on Source Coding using Huffman Coding done by Ajay Yadav and Manish Sinha.

--- Compression Analysis ---

Original Size: 896 bits

Compressed Size: 502 bits

Compression Ratio: 1.78

Entropy: 4.4558 bits/symbol

Average Code Length: 4.4821 bits/symbol

CONCLUSION

The Huffman coding algorithm provides an efficient method for data compression, especially for texts with varying character frequencies. In this implementation, the process of encoding and decoding the text works accurately, ensuring no loss of information.

The key findings from the analysis are:

1. **Compression Efficiency:** The Huffman coding algorithm successfully reduces the size of the input text, achieving a compression ratio of approximately 1.85, which means the compressed text occupies only about 54% of the original size.
2. **Entropy vs. Average Code Length:** The entropy value (4.2413 bits/symbol) indicates the theoretical lower bound for the number of bits needed for encoding. The average code length (4.1715 bits/symbol) is very close to this entropy, reflecting the efficiency of the algorithm. This shows that Huffman coding is almost optimal for the given text.
3. **Lossless Compression:** The text was compressed and then accurately decompressed, proving that Huffman coding is a **lossless** compression technique, preserving the integrity of the original data.

In conclusion, Huffman coding proves to be a highly effective and efficient method for text compression, achieving a good balance between compression and information retention. Its performance is close to the theoretical limit, making it a reliable choice for data compression tasks in various applications.

FUTURE SCOPE

While Huffman coding is an efficient algorithm for data compression, there are several avenues for further improvement and research:

1. **Adaptive Huffman Coding**:
    - **Current Limitation**: Traditional Huffman coding requires knowledge of the frequency distribution beforehand, which may not be available in dynamic or real-time applications.
    - **Future Scope**: Adaptive Huffman coding can be explored, where the algorithm adjusts the Huffman tree as the text is being processed, making it more suitable for streaming data or applications where data distribution changes over time.
2. **Huffman Coding in Multimedia Compression**:
    - **Current Limitation**: Huffman coding is primarily used in text-based compression.
    - **Future Scope**: Research can be extended to use Huffman coding in multimedia compression, such as image, audio, and video compression, where it can be combined with other techniques (like Discrete Cosine Transform or wavelet transforms) for better performance.
3. **Comparison with Other Algorithms**:
    - **Current Limitation**: Huffman coding may not always be the most efficient in terms of compression ratio, especially for specific data types.
    - **Future Scope**: Comparing Huffman coding with other modern compression algorithms (like Arithmetic coding, LZ77, or Burrows-Wheeler Transform) could help identify when and where Huffman coding should be used for optimal performance.
4. **Parallel and Distributed Processing**:
    - **Current Limitation**: The standard Huffman coding algorithm may not be efficient for large-scale data.
    - **Future Scope**: Implementing Huffman coding on parallel or distributed computing systems could speed up the compression and decompression processes, making it more suitable for big data applications.

5. **Error-Resilient Compression**:
    - **Current Limitation**: Huffman coding does not inherently provide error resilience, making it susceptible to errors in noisy environments.
    - **Future Scope**: Integrating error-correction techniques with Huffman coding could be beneficial for applications in communication systems or storage devices where data integrity is critical.
6. **Integration with Machine Learning**:
    - **Future Scope**: Machine learning techniques could be used to predict the most optimal Huffman tree based on the input data, potentially improving compression efficiency for specific types of data or applications.

By addressing these areas, Huffman coding could become more versatile and adaptable to a wider range of real-world applications, ensuring its continued relevance in the evolving field of data compression.

BIBLIOGRAPHY

- **Huffman, D. A.** (1952). *A Method for the Construction of Minimum-Redundancy Codes*. Proceedings of the IRE.
    - Introduces Huffman coding.
- **Salomon, D.** (2007). *Data Compression: The Complete Reference*. Springer.
    - Comprehensive guide on data compression.
- **Cover, T. M., & Thomas, J. A.** (2006). *Elements of Information Theory*. Wiley.
    - In-depth explanation of information theory.
- **Sayood, K.** (2012). *Introduction to Data Compression*. Morgan Kaufmann.
    - Overview of compression techniques.
- **Ziv, J., & Lempel, A.** (1977). *A Universal Algorithm for Sequential Data Compression*. IEEE IT.
    - Introduces Lempel-Ziv compression.
- **Li, X., & Shao, Z.** (2011). *Application of Huffman Coding in Multimedia*. IJCSNS.
    - Discusses Huffman coding in multimedia.
- **Witten, I. H., et al.** (1987). *Arithmetic Coding for Data Compression*. ACM.
    - Focus on arithmetic coding.